

Beginner Build Guide

Low-cost hardware + software for the Intelligent Classroom Energy Monitor

For a team with **no hardware experience**. Every wire, every setting, every check. Works together with the 'Prototype Edition' document and the code package **classroom-energy-code.zip**.

What you build	A 12 V bench 'classroom': LED bank (light), DC fan (fan), second DC fan (projector). Three power sensors measure each appliance. An ESP32 sends the readings to your laptop, which stores them as your own dataset, trains AI models, predicts future usage in real time and flags anomalies.
New money needed	About ₹500 - ₹850 (3 power-sensor modules + a few cheap parts). Everything else is already in the IEEE SIG lab list.
Time	Hardware: 2-3 working sessions. Data collection: about 24 hours of unattended running. Training and demo: 1 session.
Voltage	Only 12 V DC and 5 V. No mains (230 V) anywhere. This is safe to touch, but follow the rules in Part 3.

Contents

- **Part 1** Shopping list (what to buy, what to take from the lab, what NOT to buy)
- **Part 2** The big picture and how the circuit works
- **Part 3** Electronics in 10 minutes (read this once) and safety rules
- **Part 4** Software setup on your laptop
- **Part 5** Build in 9 small stages, testing after each one
- **Part 6** Run the system: data collection, training, live prediction, demo
- **Part 7** Troubleshooting and the final wiring checklist

Part 1. Shopping List (Minimum Cost)

1.1 Buy these (the only things not in the lab)

Item	Qty	Approx. price	Why you need it
INA219 current/power sensor module (I2C, with screw terminal, ask for 'pins soldered')	3	₹150-250 each (₹450-750)	A tiny 'watt meter' chip. One per appliance. This is the only essential purchase.
1N4007 diode	3	₹1-3 each	Protects the MOSFET from the fan's voltage spike (2 used, 1 spare)
DC barrel-jack to screw-terminal adapter	1	₹20-40	ONLY if the 12 V 5 A supply ends in a round plug. Skip if it has bare wires.
Micro-USB (or Type-C, check your ESP32) data cable	1	₹60-100	ONLY if you do not own one. Must be a data cable, not charge-only.

Prices are typical Indian online/market estimates and change; check current listings. Total new money: about **₹500 - ₹850**.

1.2 Take these from the lab (already owned: item numbers from the lab list)

Part	Lab item	Qty	Used for
ESP32 microcontroller	#44 (has 2)	1	The brain: reads sensors, sends Wi-Fi data, controls loads
12 V 5 A power supply	#42	1	Powers the fans and LEDs
12 V DC fan	#115 (has 2)	2	Classroom fan + projector stand-in
LED assortment kit (any colour)	#36	8 LEDs	Classroom light (8 LEDs)
1k ohm resistors	#104 (has 50)	10	8 for LEDs + 2 for MOSFET gates
IRLZ44N MOSFET	#114 (has 2)	2	Electronic on/off tap with dimming (light, fan)
2-channel 5 V relay module	#26 or #129	1	ON/OFF switch for the projector load
LDR sensor module	#35 (has 2)	1	Brightness sensor
DHT11 temperature + humidity	#7 or #108	1	Room temperature and humidity

Part	Lab item	Qty	Used for
Breadboard	#31 (has 2)	2	Plug-in wiring without soldering
Jumper wires (M-F, F-F, M-M)	#103	1 set	Connections
Single-strand wire	#117	2 m	Wires into screw terminals (strip the ends)
Digital multimeter	#59	1	Checking voltages and calibrating
Soldering station + wire cutter	#58, #101	-	Only for 2 tiny solder blobs on the INA219 address pads (Stage 2)
Laptop / PC	yours	1	Runs the backend and AI. A Raspberry Pi is NOT required.

1.3 Do NOT buy (not needed, saves money)

- PIR / people sensors (the project does not detect people), PZEM-004T and CT clamps (those are for 230 V mains, only for a later real-classroom version), Raspberry Pi, OLED, buck converter, extra ESP32 or Arduino boards, LED strips.

1.4 Budget fallback: only ONE INA219 (about ₹150-250 total)

If money is a hard limit, buy one INA219 and put it on the main +12 V feed so it measures the **total** of all appliances. The firmware has a switch `SINGLE_SENSOR_MODE` that then **estimates** each appliance's watts from the command being applied. Forecasting and the waste rule still work. The cost: appliance-level numbers become estimates, so appliance attribution and the 'estimate appliances from total' AI module are weaker to defend in front of faculty. We recommend 3 sensors.

Part 2. The Big Picture

Each appliance is one path: **+12 V → INA219 (measures) → appliance → on/off switch → GND**. The ESP32 reads each INA219 over a shared 2-wire bus (I2C), and also controls the on/off switches. The laptop tells the ESP32 what to switch (a virtual classroom timetable), stores the readings, trains the AI and predicts the future.

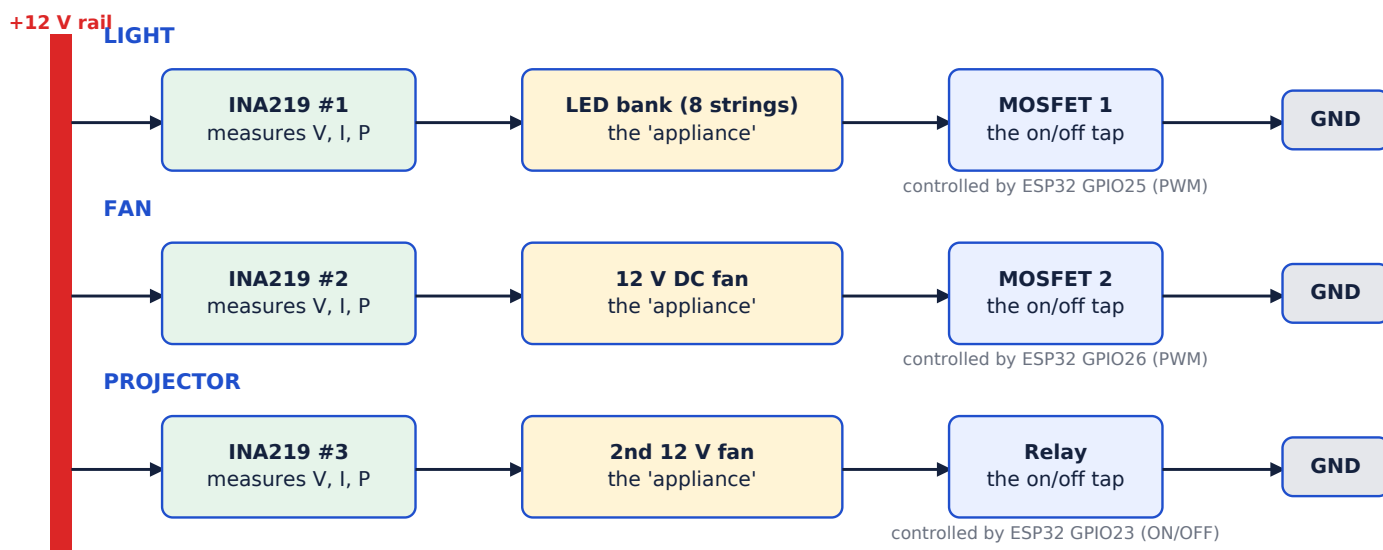


Figure 1: The three power paths. The INA219 sits 'in line' with the appliance, like a water-flow meter in a pipe.

Why this design is cheap and safe

- The appliances are tiny (about 1-3 watts each) so no expensive sensors are needed.
- The AI does not care whether a fan is 2 W or 200 W. It learns patterns of use. The same software works later on a real classroom with isolated mains meters (same JSON, same AI).
- Light sensor note:** the LDR measures brightness, not electricity. It is only an extra feature. Electricity is measured by the INA219 (volts x amps = watts).

Part 3. Electronics in 10 Minutes

Word	Meaning (simple)	On our bench
Voltage (V)	Electrical 'pressure', like water pressure	12 V for loads, 3.3 V for sensors, 5 V for the relay coil
Current (A, mA)	How much electricity flows, like litres per second	LEDs about 75 mA, a fan about 100-250 mA
Power (W)	Voltage x current = how much energy is used per second	Total of the bench: roughly 1 - 7 W
GND (ground)	The common 'return' wire / zero reference	Every GND must be joined together (supply, ESP32, sensors)
MOSFET	An electronic tap. A small signal on the Gate opens/closes a big path (Drain to Source)	Lets the ESP32 turn the LEDs/fan on and dim them
PWM	Switching on/off very fast. 30% on-time = about 30% brightness/speed	Dimming the light, setting fan speed
I2C	A 2-wire bus (SDA, SCL) many chips can share, each with its own address	ESP32 talks to the 3 INA219 over the same two wires
Flyback diode	Stops the voltage spike a motor makes when switched off	1N4007 across each fan

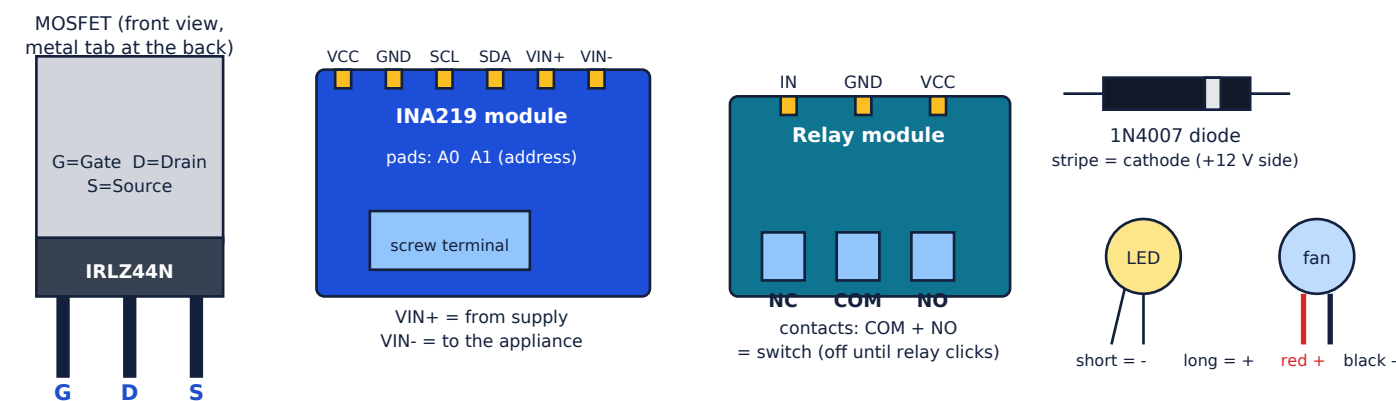


Figure 2: Parts you will wire. Check pin labels on YOUR module; they are printed on the board.

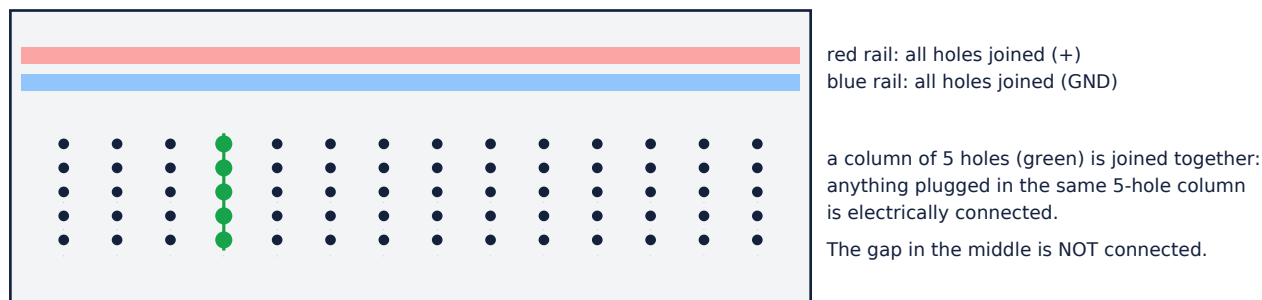


Figure 3: A breadboard. Columns of 5 holes are joined; the long coloured rails run the full length.

Safety rules (read every one)

1. Only 12 V DC and 5 V. **Never connect 230 V mains** to the breadboard, ever.
2. Unplug the 12 V supply before changing any wire. Wire first, power on after you re-check.
3. **Polarity matters:** red/+ and black/- must not be swapped on the INA219 VIN+/VIN-, diode, LED and fan. Wrong polarity can kill parts.
4. Never connect +12 V directly to GND, and never connect 12 V to an ESP32 pin. The ESP32 pins only take 3.3 V.
5. If anything gets hot, smells, or smokes, unplug immediately.
6. Use a data USB cable for the ESP32 and power the ESP32 from USB (laptop). Do not power it from the 12 V supply.

Using the multimeter

- **Voltage:** dial on V with the straight line (DC), red probe in the 'V' jack, black in 'COM'. Touch the probes across the two points (in parallel).

- **Current:** you must break the wire and put the meter in the middle (in series): dial on the 200 mA range, red probe moved to the 'mA' jack. **Move the red probe back to the V jack afterwards;** leaving it in the mA jack and then measuring voltage can blow the meter's fuse.

Part 4. Software Setup on Your Laptop (Windows steps)

Do this before building so you can test as you go.

1. **Arduino IDE 2.x:** download from arduino.cc, install with defaults.
2. **USB driver:** your ESP32 board uses a CP2102 chip (the lab list says 'CP2102 driver'). Install the 'CP210x Universal Windows Driver' from Silicon Labs. (If your board has a CH340 chip, install the CH340 driver instead.) Plug the ESP32 in; a new COM port should appear in Device Manager.
3. **ESP32 board package:** Arduino IDE → File → Preferences → in 'Additional boards manager URLs' paste https://espressif.github.io/arduino-esp32/package_esp32_index.json → OK. Then Tools → Board → Boards Manager → search 'esp32' → install '**esp32 by Espressif Systems**'.
4. Select the board: Tools → Board → esp32 → **ESP32 Dev Module**. Tools → Port → choose your COM port.
5. **Libraries** (Sketch → Include Library → Manage Libraries, search and Install each): **Adafruit INA219** (accept to install dependencies), **DHT sensor library** by Adafruit (accept dependencies), **ArduinoJson** by Benoit Blanchon (version 7.x).
6. **Python 3.12:** download from python.org. On the first installer screen **tick 'Add python.exe to PATH'**.
7. **Project:** unzip **classroom-energy-code.zip** to a simple folder like C:\classroom-energy.
8. **Python packages:** open the folder, click the address bar, type cmd, press Enter, then run:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Tip: When uploading a sketch, if the IDE shows 'Connecting.....' and fails, hold the **BOOT** button on the ESP32 until upload starts, then release.

Test the software now, with no hardware (uses simulated data)

```
uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

```
# open a SECOND terminal in the same folder:
venv\Scripts\activate
python tools\fake_esp32.py --steps 300 --interval 0.2
```

Open **<http://localhost:8000>**. You should see the dashboard filling with data. Windows may ask to allow Python through the firewall: click **Allow (private networks)**. Stop the server with Ctrl+C. **Then delete** data\classroom_energy.db so the simulated rows do not mix with your real data.

Part 5. Build in 9 Small Stages

Rule: finish and test each stage before starting the next. Power off (unplug the 12 V supply) whenever you change wiring. Breadboard A = 12 V power + loads. Breadboard B = ESP32 + 3.3 V sensors. Only the GND wire links the two boards.

Stage 1. Meet the ESP32 (no wiring)

1. Plug the ESP32 into the laptop with the USB data cable.
2. Open `firmware/tests/01_i2c_scanner` in Arduino IDE, click Upload (arrow).
3. Tools → Serial Monitor, set speed to **115200**. You should see 'Scanning...' and 'NOTHING found'. That is correct for now: it proves upload and USB work.

Stage 2. Prepare the three INA219 modules

Each INA219 needs its own I2C address, set by two solder pads marked **A0** and **A1** on the board. Leave the pads alone on module 1.

Module	Pads to bridge with a small blob of solder	Address
#1 Light	none	0x40
#2 Fan	A0 only	0x41
#3 Projector	A1 only	0x44

- If the module came with loose pin headers, solder them to the 6 holes (VCC GND SCL SDA VIN+ VIN-) so it fits the breadboard. Ask the lab assistant to help once; it is 5 minutes.
- Write 1, 2, 3 on each module with a marker.
- The scanner in Stage 3 tells you the real address. Whatever it shows, type it into the matching line of the firmware (`Adafruit_INA219 inaLight(0x40), inaFan(0x41), inaProj(0x44)`).

Stage 3. First real measurement: one INA219 + the LED bank

Make the LED bank (8 strings). One string = a 1k resistor in series with one LED. Use the lower rails of Breadboard A: the lower red rail is 'LED+', the lower blue rail is 'LED-'. For each of 8 strings: resistor from the LED+ rail to a free column, LED (long leg to the resistor column, short leg) to the LED- rail. Each LED uses about 10 mA, total about 75 mA (about 0.9 W).

#	Connect	To
1	12 V supply +	Breadboard A top red rail (+12 V)
2	12 V supply -	Breadboard A top blue rail (GND)
3	Breadboard A GND rail	Breadboard B GND rail (long black jumper)
4	ESP32 3V3 pin	Breadboard B red rail (3.3 V); ESP32 GND to B blue rail
5	INA219 #1 VCC	B red rail (3.3 V)
6	INA219 #1 GND	B blue rail (GND)
7	INA219 #1 SDA	ESP32 GPIO21
8	INA219 #1 SCL	ESP32 GPIO22
9	A +12 V rail	INA219 #1 VIN+ (screw terminal, single-strand wire)
10	INA219 #1 VIN-	A lower red rail (LED+)
11	A lower blue rail (LED-)	A GND rail (for now the LEDs go straight to GND; the MOSFET comes in Stage 4)

1. Upload `firmware/tests/01_i2c_scanner` again. You should now see **device at 0x40**.
2. Turn on the 12 V supply: the LEDs light. Upload `02_ina219_test` and open the Serial Monitor.
3. **Expected:** LIGHT V about 11.8-12.1, I about 60-90 mA, P about 0.7-1.1 W. The 0x41/0x44 'missing' messages are fine for now.
4. **Check with the multimeter:** remove the wire from INA219 VIN- to the LED+ rail, connect the meter (mA mode) between them. The multimeter and Serial Monitor current should agree within about 5%. Restore the wire and the red probe.

Stage 4. Make the light dimmable (MOSFET 1)

Power OFF. The MOSFET goes between the LED bank and GND. Hold it with the metal tab at the back and pins down: left = **Gate**, middle = **Drain**, right = **Source**.

#	Change	Detail
1	Remove the LED- to GND wire from step 11	LED- rail now goes to the MOSFET instead
2	A lower blue rail (LED-)	→ MOSFET 1 Drain (middle pin)
3	MOSFET 1 Source (right pin)	→ A GND rail
4	MOSFET 1 Gate (left pin)	→ ESP32 GPIO25
5	1k resistor	from MOSFET 1 Gate to GND (keeps it off while the ESP32 boots)

1. Upload 03_pwm_relay_test. The LEDs step through 0, 25, 50, 75, 100% brightness (the fan and relay are not wired yet; that is fine).
2. Run 02_ina219_test again while dimmed: the LIGHT power should rise and fall with brightness.

Stage 5. Add the fan (INA219 #2 + MOSFET 2 + diode)

#	Connect	To
1	INA219 #2 VCC / GND / SDA / SCL	B 3.3 V rail / B GND rail / GPIO21 / GPIO22 (SDA and SCL share the same ESP32 pins as #1)
2	A +12 V rail	INA219 #2 VIN+
3	INA219 #2 VIN-	Fan red wire (+)
4	Fan black wire (-)	MOSFET 2 Drain
5	1N4007 diode	across the fan: stripe end to the fan red (+) , other end to the fan black (-)
6	MOSFET 2 Source	A GND rail
7	MOSFET 2 Gate	ESP32 GPIO26, plus a 1k resistor from Gate to GND

1. Re-run 01_i2c_scanner: you should see 0x40 and 0x41.
2. Run 03_pwm_relay_test: the fan speeds up in steps. Run 02_ina219_test: FAN power rises with speed (about 1-3 W at full speed).
3. If the fan only hums or stalls at low levels, that is normal for some fans: the real firmware lets the AI scenario use 40-100%.

Stage 6. Add the projector load (INA219 #3 + relay)

The projector stand-in is the second 12 V fan, switched ON/OFF by the relay (a projector is a steady load).

#	Connect	To
1	INA219 #3 VCC / GND / SDA / SCL	B 3.3 V / GND / GPIO21 / GPIO22
2	A +12 V rail	INA219 #3 VIN+
3	INA219 #3 VIN-	Relay COM (screw terminal)
4	Relay NO	2nd fan red (+)
5	2nd fan black (-)	A GND rail
6	1N4007 diode	across the 2nd fan, stripe end to red (+)
7	Relay module VCC	ESP32 VIN pin (5 V, available when powered by USB)
8	Relay module GND	GND (ESP32 GND)
9	Relay module IN1	ESP32 GPIO23

1. Scanner should now show 0x40, 0x41 and 0x44 (if you see 0x45 or another number, use it in the firmware).
2. Run 03_pwm_relay_test: you hear the relay click ON then OFF. If it is ON when the sketch says OFF, change RELAY_ACTIVE_LOW from 1 to 0 in the sketch (and later in the main firmware).
3. While the relay is ON, 02_ina219_test shows PROJ power about 1-3 W.

Stage 7. Add the LDR and DHT11

#	Connect	To
1	LDR module VCC / GND	B 3.3 V / GND

#	Connect	To
2	LDR module AO (analog out)	ESP32 GPIO34
3	DHT11 module + / -	B 3.3 V / GND
4	DHT11 module S (signal)	ESP32 GPIO4

Upload 04_dht_ldr_test. The temperature and humidity should look like your room. The LDR number should change when you cover it with your hand. (If DHT shows nan, check the three wires and that the module is on 3.3 V.)

Stage 8. Calibrate against the multimeter (15 minutes, important for credibility)

1. Run 02_ina219_test with a load ON (use 03 to set a level, or just let the light run).
2. Put the multimeter in series (mA mode) with that appliance and write down meter mA and INA219 mA at three levels (low, medium, full). Repeat for light, fan, projector.
3. Compute **factor = meter mA / INA219 mA** (average of the three). If every factor is between 0.95 and 1.05 leave them at 1.0.
4. Otherwise type the factors into the main firmware: CAL_LIGHT, CAL_FAN, CAL_PROJ. Keep a table of these numbers for your report.

Stage 9. Connect to Wi-Fi and the backend

1. **Use a Wi-Fi network where devices can talk to each other.** The ESP32 only works on 2.4 GHz. College Wi-Fi often blocks device-to-device traffic, so the easiest option is your **phone hotspot (2.4 GHz band)** with both the laptop and ESP32 connected to it.
2. On the laptop, find its IP address: open cmd and run ipconfig; note the **IPv4 Address** (like 192.168.43.25).
3. Start the server: `uvicorn backend.main:app --host 0.0.0.0 --port 8000`. Allow Python in the Windows firewall when asked (private networks).
4. Open 05_wifi_server_test, edit WIFI_SSID, WIFI_PASS and SERVER (`http://YOUR_IP:8000`), upload. **Expected:** 'HTTP 200' and a JSON command every 3 seconds. If not: see Part 7.
5. Open firmware/esp32_energy, edit the same three lines (and the calibration factors), upload. In the Serial Monitor you see 'POST {...}', 'REPLY {...}' and 'COMMAND ...' every 10 seconds, and the fans/LEDs follow the virtual timetable. The dashboard shows the live numbers.
6. **Hardware is finished.** Take a photo of the setup for your report.

Part 6. Run the System

6.1 What the system is doing

Every 10 real seconds the ESP32 sends one reading. That is one **virtual 10-minute classroom interval**: the backend runs a virtual timetable 60 times faster than real time, so **one virtual day takes 24 real minutes**. The backend replies with the next command (light %, fan %, projector on/off) following that timetable: empty at night and weekends, classes 09:00 to 16:00 on weekdays, lunch break, random holidays, hot days, and about 3% deliberately injected, labelled faults (fan stuck on at night, projector left on, lights on in an empty room).

Real-time running	Virtual time	Rows collected
24 minutes	1 day	144
12 hours	30 days	about 4,300
24 hours	60 days	about 8,600 (recommended)

6.2 Collect your own dataset

1. Delete any old database: `data\classroom_energy.db` and `ml\models\*.json`.
2. Start the server and power the ESP32 and the 12 V supply. Confirm rows are arriving on the dashboard.
3. Laptop settings: **Power & sleep** → **Never sleep** while plugged in; keep the terminal open; keep the phone hotspot on and charged (or use a normal router).
4. Let it run about 24 hours (it can be several sessions; it continues where it stopped). Do not move wires during collection.
5. Download the dataset any time: `http://localhost:8000/api/export.csv` (this is your own dataset for the report).

6.3 Train the AI

```
python ml\train.py
```

This uses a strict **chronological split** (70% train, 15% validation, 15% test) and prints two tables: forecast accuracy (naive baseline, linear regression, random forest, XGBoost) and anomaly detection (precision/recall of the residual rule, the waste rule, and both combined). It saves the models into `ml\models`; the running server picks them up automatically.

Honest reporting: in our software test with **simulated** data (about 8,500 rows) XGBoost beat the 'next = now' baseline clearly (R-squared about 0.97 vs 0.87), but with only 3,000 rows it did not. Collect enough data (6,000+ rows) and report only numbers from **your own hardware dataset**, never from simulated data.

6.4 Live prediction and anomaly detection

- After training, the dashboard shows **Predicted next 10 min**, the actual-vs-predicted chart, a **Forecast next hour**, appliance power lines and the **Anomaly log** (which appliance caused it, how many sigma, which rule fired).
- Two rules run together: the **residual rule** (actual vs AI expectation beyond 3 standard deviations from validation data) and the **waste rule** (power above 0.3 W while the timetable says the room is not in use).
- Tick **Auto cut-off** on the dashboard: when an anomaly is detected the next command from the server turns all loads off (closed-loop energy saving).

6.5 Demo script for faculty (5 minutes)

1. Show the bench and explain: LED = classroom light, fan = fan, second fan = projector, INA219 = per-appliance meter.
2. Show the live dashboard following the virtual timetable, and the predicted line tracking the actual line.
3. Press **Projector left ON** on the dashboard: the projector fan runs when it should not. The anomaly appears, naming the projector and its sigma.
4. Tick **Auto cut-off** and repeat: the system switches the loads off by itself and reports the saving.
5. Show the results table from `train.py`, the downloaded CSV, and one slide on scaling to a real classroom with isolated mains meters.

Part 7. Troubleshooting

Problem	Likely cause	Fix
Upload fails ('Connecting.....')	Bad cable, wrong port, needs boot mode	Use a data cable; select the right COM port; hold BOOT while uploading; try another USB port
No COM port appears	Driver missing / charge-only cable	Install CP210x (or CH340) driver; change cable
Scanner finds nothing	SDA/SCL swapped or loose, no 3.3 V	SDA = GPIO21, SCL = GPIO22; check VCC to 3V3 and GND; re-seat wires
Scanner finds an unexpected address	Address pads bridged differently	Use the address it shows in the firmware (inaLight/inaFan/inaProj lines)
INA219 current reads 0	Appliance off, or INA219 wired backwards, or VIN+/VIN- swapped	VIN+ must come from +12 V, VIN- must go toward the appliance
Negative or wildly wrong current	VIN+ and VIN- swapped / no common GND	Swap them; make sure supply GND, ESP32 GND and all module GNDs are joined
LEDs never switch off or fan always on	MOSFET legs wrong way	Left Gate, middle Drain, right Source (tab at back); add the gate-to-GND resistor
MOSFET gets hot	Wrong pinout or not fully on	Power off, re-check pins; use an IRLZ44N (logic-level), not a normal IRF type
Relay inverted	Active-low module	Flip RELAY_ACTIVE_LOW (1 or 0)
DHT11 shows nan	Wiring or wrong pin	Check S to GPIO4 and 3.3 V; wait 2 s between reads
HTTP error -1 / cannot reach server	Wrong laptop IP, different Wi-Fi, firewall, client isolation	Same network; recheck ipconfig; allow Python in the firewall; use phone hotspot (2.4 GHz)
Open http://IP:8000 from phone fails	Server started without --host 0.0.0.0	Restart: <code>uvicorn backend.main:app --host 0.0.0.0 --port 8000</code>
Dashboard shows 'no model yet'	You have not trained yet	Collect 4,000+ rows, then run <code>python ml\train.py</code>
Fan twitches at low speed	Brushless fans dislike low PWM	Normal; keep fan levels at 40% or higher in your scenario

Final wiring checklist (tick before collecting data)

- Supply +12 V reaches VIN+ of all three INA219. VIN- goes only toward its own appliance.
- Every appliance ends in a switch (MOSFET or relay) that goes to GND.
- One common GND joins: supply -, Breadboard A, Breadboard B, ESP32, all modules, MOSFET sources, relay GND.
- ESP32 3V3 feeds INA219 VCC, LDR VCC, DHT VCC. Relay VCC is on ESP32 VIN (5 V).
- Pin map: SDA 21, SCL 22, light MOSFET 25, fan MOSFET 26, relay 23, LDR 34, DHT11 4.
- Scanner shows three INA219 addresses; multimeter calibration done; Wi-Fi test returns HTTP 200.

Scaling to a real classroom later: replace each INA219 by an isolated mains meter (for example PZEM-004T on each circuit, installed by a qualified electrician). The ESP32 payload, database, features and AI stay the same.